

CS5004, Spring 2026

Lab 9: Design Patterns

Adapted from lab 09 by Tamara Bonaci = tbonacin@northeastern.edu.

Lab Problem 1: Smart Home Control System

You are building a smart home control system that manages various devices and provides a unified interface for users.

Requirements

Implement a system using the following design patterns:

Patterns to implement: (read about the patterns we did not cover in class before implementing)

- **Singleton:** `SmartHomeController` (only one instance should exist)
- **Factory Method:** `DeviceFactory` to create different smart devices
- **Adapter:** `LegacyThermostatAdapter` to integrate an old thermostat with incompatible interface
- **Facade:** `SmartHomeFacade` to provide a simple interface for complex operations

Classes to implement:

```
// Device interface
interface SmartDevice {
    void turnOn();
    void turnOff();
    String getStatus();
}

// Legacy thermostat with incompatible interface
class LegacyThermostat {
    public void activate() { /* implementation */ }
    public void deactivate() { /* implementation */ }
    public int getCurrentTemp() { /* implementation */ }
}
```

Tasks:

1. Create `SmartLight` and `SmartSpeaker` classes implementing `SmartDevice`
2. Implement `SmartHomeController` as a singleton that manages a list of devices
3. Create `DeviceFactory` with a factory method to create devices by type ("light", "speaker")
4. Implement `LegacyThermostatAdapter` to make the legacy thermostat work with your `SmartDevice` interface
5. Create `SmartHomeFacade` that provides methods like `activateNightMode()` and `leaveHome()` that control multiple devices at once

Expected Deliverables:

A main method demonstrating:

- Creating devices through the factory
- Adding devices to the singleton controller
- Using the adapter for the legacy thermostat
- Using the facade for complex operations

Lab Problem 2: Game Character Creation System

Develop a character creation system for an RPG game that handles complex character builds and provide different representations.

Requirements

Implement a system using the following design patterns:

Patterns to implement: (read about the patterns we did not cover in class before implementing)

- **Builder:** `CharacterBuilder` for step-by-step character creation
- **Prototype:** Characters can be cloned with modifications
- **Decorator:** Add equipment/abilities that modify character stats
- **Factory Method:** `CharacterFactory` to create different character archetypes

Base classes provided:

```
// Base character class
abstract class GameCharacter implements Cloneable {
    protected String name;
    protected int health;
    protected int attack;
    protected int defense;
    // Constructor and basic methods
    public abstract GameCharacter clone();
    public abstract String getDescription();
}

// Equipment interface for decorator
interface Equipment {
    int getAttackBonus();
    int getDefenseBonus();
    String getDescription();
}
```

Tasks:

1. Create concrete character classes: `Warrior`, `Mage`, `Archer` extending `GameCharacter`
2. Implement `CharacterBuilder` with methods like `setName()`, `setHealth()`, `addEquipment()`, `build()`
3. Make characters cloneable (implement the clone method properly)
4. Create decorator classes: `WeaponDecorator` and `ArmorDecorator` that wrap characters and modify their stats
5. Implement `CharacterFactory` with factory methods for creating pre-configured character templates
6. Create `CharacterManager` class to demonstrate creating, cloning, and decorating characters

Tasks related to design patterns:

1. Implement the builder pattern for flexible character creation
2. Enable character cloning with the prototype pattern
3. Create equipment decorators that enhance character abilities
4. Use factory method to create character templates ("tank", "dps", "support")
5. Main method showing character creation, cloning, and decoration

Expected Deliverables:

A main method demonstrating:

- Creating characters through factory methods with different configurations
- Cloning existing characters and modifying them
- Decorating characters with equipment
- Displaying character stats and descriptions